



Audit de l'application existante

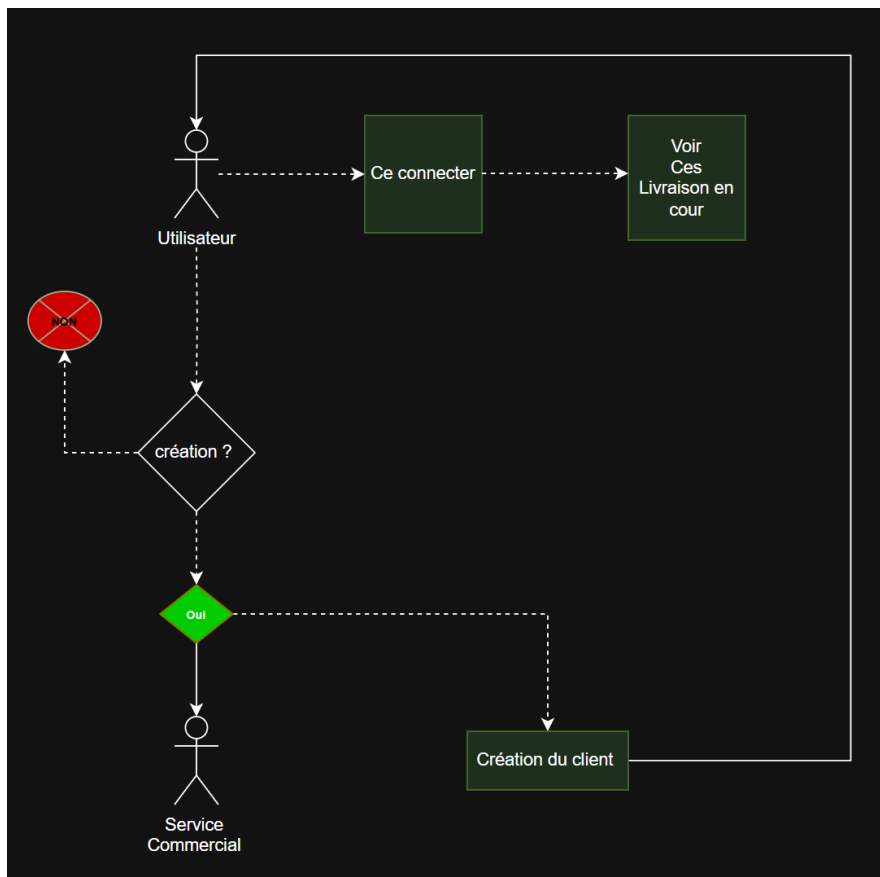
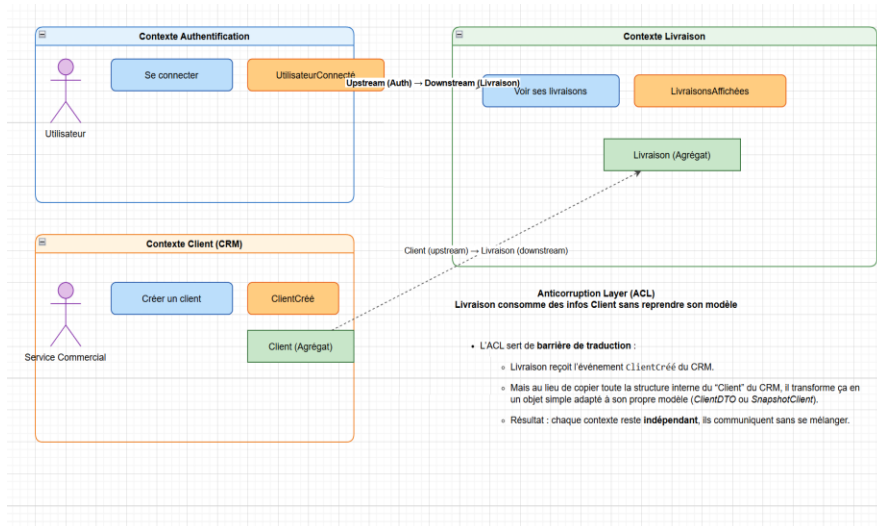
1. Contexte

- *LiVrai est une société de 150 employés qui propose un service de livraison de marchandises en grande quantité*
- *À quoi sert l'application.*

2. Fonctionnalités

La liste des fonctionnalités de l'application sont les suivants,

- *La connexion et l'authentification*
 - *Se connecter*
- *Le contexte client CRM*
 - *Cree un client*
- *La livraison*
 - *Voir les livraisons*
 - *Livraison achevée*
 - *Livraison Agrégat*



Lorsque l'**utilisateur** arrive, il doit d'abord **se connecter**.
 S'il a déjà un compte, il peut alors **voir ses livraisons en cours**.
 S'il **n'a pas de compte**, la question « création ? » est posée :

- **Non** → le processus s'arrête.
- **Oui** → le **Service Commercial** est sollicité pour procéder à la **création du client**. Une fois le client créé, l'utilisateur pourra se connecter et accéder à ses livraisons.

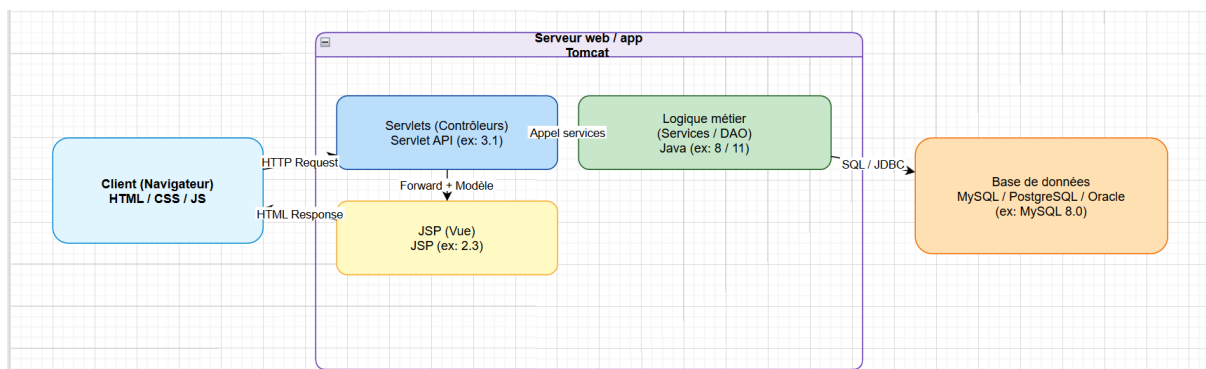
3. Expérience utilisateur

L'application Livrai offre actuellement une expérience utilisateur correcte grâce à sa rapidité et à la simplicité de ses fonctionnalités principales (connexion, suivi des livraisons, gestion des clients).

Toutefois, comme la majorité des applications JSP/Servlets, elle souffre de limites ergonomiques : rafraîchissements complets des pages, design daté et difficulté d'adaptation aux terminaux mobiles.

La maintenabilité technique réduit également la capacité à faire évoluer rapidement l'interface et, par conséquent, à améliorer l'expérience utilisateur sur le long terme.

4. Description technique



- **Client (navigateur)**
L'utilisateur interagit via une page HTML/CSS/JS. Une **requête HTTP** est envoyée au serveur.
- **Serveur web/app – Tomcat**
- **Servlets (Contrôleurs)** reçoivent la requête. Elles analysent les paramètres, valident les données et décident quoi faire ensuite.
- Elles **appellent les services** de la **logique métier**.
- **Logique métier (Services / DAO, Java)**
- Les **services** contiennent les règles fonctionnelles.
- Les **DAO** accèdent aux données. Pour lire/écrire, ils utilisent **SQL/JDBC** vers la base de données.
- **Base de données**
Moteur (ex. MySQL 8.0). Il exécute les requêtes SQL et renvoie les résultats au serveur.
- **Retour serveur**
Les résultats (le "modèle") sont transmis à une **JSP (Vue)** qui génère la **réponse HTML**.
- **Réponse au client**
Tomcat renvoie la page HTML au navigateur, qui l'affiche à l'utilisateur.

5. Points forts et déficiences

Les points forts actuels de l'application résident dans sa simplicité et sa rapidité d'affichage, grâce à une interface légère (technologie JSP).

En revanche, l'application présente certaines limites :

Points forts possibles

- *Simplicité d'utilisation : interface claire, peu de fonctionnalités → moins de formation nécessaire.*
- *Coût faible de maintenance : application légère qui ne nécessite pas beaucoup de ressources serveurs.*
- *Technologie éprouvée : JSP/Servlet est stable et bien documenté, donc peu de surprises en fonctionnement.*

Déficiences possibles

- **Expérience utilisateur limitée** : interface peu moderne, pas d'ergonomie avancée (comparée à Angular/React).
- **Évolutivité fonctionnelle réduite** : ajouter de nouvelles fonctionnalités prend du temps (architecture monolithique).
- **Dépendance technologique** : risque de manque de développeurs compétents sur JSP/Servlet (technologie vieillissante).
- **Intégration difficile** : plus compliqué de connecter l'appli à des services externes (API REST, microservices).
- **Tests limités** : absence de couverture de tests automatisés moderne (JUnit5, Cypress, etc.), ce qui augmente le risque d'erreurs.

6. Conclusion

L'application Livrai répond aujourd'hui aux besoins essentiels de suivi des livraisons et de gestion des clients grâce à sa simplicité et sa rapidité. Ces qualités en font un outil opérationnel immédiatement utilisable.

Toutefois, plusieurs limites structurelles apparaissent : une interface peu ergonomique, des difficultés d'évolution (scalabilité, ajout de fonctionnalités), et un socle technologique vieillissant qui pose des questions de sécurité et de pérennité.

Pour l'avenir, il sera donc nécessaire d'envisager une **évolution vers une architecture plus moderne** (ex. Spring Boot, Angular, API REST) afin de garantir :

- une meilleure expérience utilisateur,
- une montée en charge plus simple,
- et une sécurisation renforcée de la plateforme.

En résumé, l'application actuelle joue bien son rôle à court terme, mais une modernisation progressive est fortement recommandée pour assurer la continuité et la compétitivité du service